

LAB TASK#5

NAME: MASHAAL ISHTIAQ

ROLL NO: P22-9326

SECTION: BCS-4C

QUESTION#1

CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Product {
public:
int id;
string name;
double price;
int quantity;

// Constructor with member initializer list
Product(int id, string name, double price, int quantity)
: id(id), name(name), price(price), quantity(quantity) {}
};

class Node {
public:
Product data;
Node* prev;
Node* next;

// Constructor with member initializer list
Node(int id, string name, double price, int quantity)
: data(id, name, price, quantity), prev(nullptr), next(nullptr) {}
};

class Link_List {
private:
Node* head;

public:
Link_List() : head(nullptr) {}

~Link_List() {
while (head != nullptr) {
Node* temp = head;
head = head->next;
delete temp;
}
}

void Add_Product(int id, string name, double price, int quantity) {
Node* newProduct = new Node(id, name, price, quantity);

if (head == nullptr) {
```

```
head = newProduct;
return;
}

Node* curr = head;
Node* prev = nullptr;

while (curr != nullptr && curr->data.id < id) {
prev = curr;
curr = curr->next;
}

if (prev == nullptr) {
newProduct->next = head;
head->prev = newProduct;
head = newProduct;
} else {
prev->next = newProduct;
newProduct->prev = prev;
newProduct->next = curr;

if (curr != nullptr) {
curr->prev = newProduct;
}
}
}

void Remove_Product(int id) {
if (head == nullptr) {
cout << "Inventory is empty." << endl;
return;
}

Node* curr = head;
Node* slow = nullptr;

while (curr != nullptr && curr->data.id != id) {
slow= curr;
curr = curr->next;
}

if (curr == nullptr) {
cout << "Product with ID " << id << " not found in inventory." << endl;
return;
}

if (slow == nullptr) {
head = head->next;
if (head != nullptr) {
head->prev = nullptr;
}
} else {
```

```
slow->next = curr->next;
if (curr->next != nullptr) {
curr->next->prev = slow;
}
}

delete curr;
cout << "Product removed successfully." << endl;
}

void Display_Products() {
if (head == nullptr) {
cout << "Inventory is empty." << endl;
return;
}

Node* curr = head;
while (curr != nullptr) {
cout << "ID: " << curr->data.id << ", Name: " << curr->data.name << ", Price: " << curr-
>data.price << ", Quantity: " << curr->data.quantity << endl;
curr = curr->next;
}
}

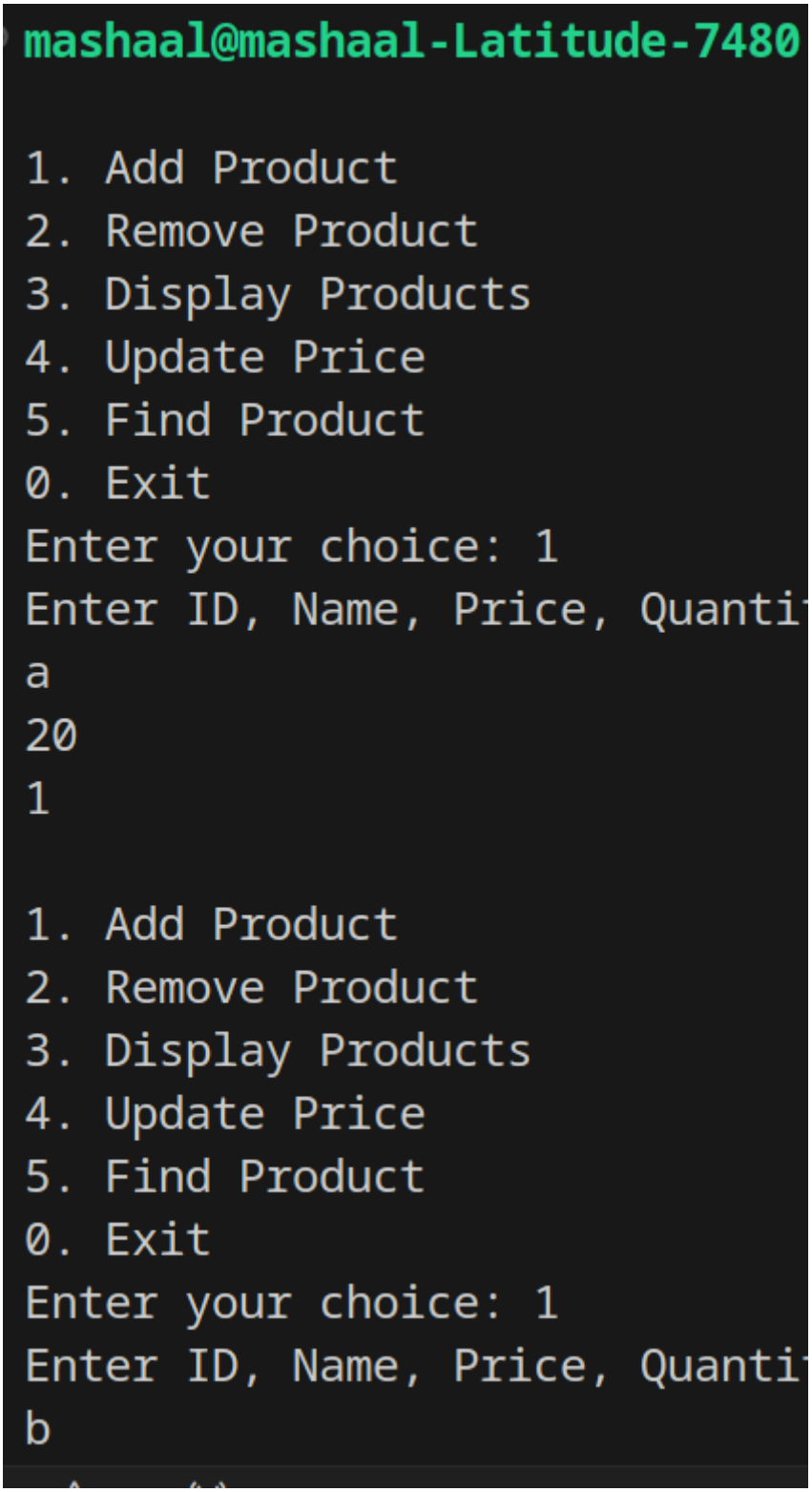
void Update_Price(int id, double newPrice) {
Node* curr = head;
while (curr != nullptr) {
if (curr->data.id == id) {
curr->data.price = newPrice;
cout << "Price updated for product with ID " << id << "." << endl;
return;
}
curr = curr->next;
}
cout << "Product with ID " << id << " not found in inventory." << endl;
}

void Find_Product(int id) {
Node* curr = head;
while (curr != nullptr) {
if (curr->data.id == id) {
cout << "Product found:" << endl;
cout << "ID: " << curr->data.id << ", Name: " << curr->data.name << ", Price: " << curr-
>data.price << ", Quantity: " << curr->data.quantity << endl;
return;
}
curr = curr->next;
}
cout << "Product with ID " << id << " not found in inventory." << endl;
}
};
```

```
int main() {
Link_List inventory;
int choice;
do {
cout << "\n1. Add Product\n2. Remove Product\n3. Display Products\n4. Update Price\n5. Find Product\n0. Exit\nEnter your choice: ";
cin >> choice;

switch (choice) {
case 1: {
int id, quantity;
double price;
string name;
cout << "Enter ID, Name, Price, Quantity: ";
cin >> id >> name >> price >> quantity;
inventory.Add_Product(id, name, price, quantity);
break;
}
case 2: {
int id;
cout << "Enter ID of product to remove: ";
cin >> id;
inventory.Remove_Product(id);
break;
}
case 3: {
inventory.Display_Products();
break;
}
case 4: {
int id;
double newPrice;
cout << "Enter ID of product and new price: ";
cin >> id >> newPrice;
inventory.Update_Price(id, newPrice);
break;
}
case 5: {
int id;
cout << "Enter ID of product to find: ";
cin >> id;
inventory.Find_Product(id);
break;
}
case 0:
cout << "Exiting program." << endl;
break;
default:
cout << "Invalid choice. Please try again." << endl;
break;
}
} while (choice != 0);
```

return 0;
}


mashaal@mashaal-Latitude-7480
1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 1
Enter ID, Name, Price, Quanti
a
20
1
1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 1
Enter ID, Name, Price, Quanti
b

```
Enter your choice: 1
Enter ID, Name, Price, Quantity: 2
b
30
2

1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 3
ID: 1, Name: a, Price: 20, Quantity: 1
ID: 2, Name: b, Price: 30, Quantity: 2

1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 2
Enter ID of product to remove: 1
```

```
Enter ID of product to remove: 1
Product removed successfully.

1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 3
ID: 2, Name: b, Price: 30, Quantity: 2

1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 4
Enter ID of product and new price: 2

40
Price updated for product with ID 2.
```

```
0. Exit
Enter your choice: 3
ID: 2, Name: b, Price: 40, Quantity: 2

1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 5
Enter ID of product to find: 2
Product found:
ID: 2, Name: b, Price: 40, Quantity: 2

1. Add Product
2. Remove Product
3. Display Products
4. Update Price
5. Find Product
0. Exit
Enter your choice: 0
Exiting program.
```

QUESTION#2

CODE:

```
#include<iostream>
using namespace std;

class Node {
public:
int data;
Node* next;
Node* prev;

Node(int val) {
data = val;
next = nullptr;
prev = nullptr;
}
};

class DoublyLinkList {
public:
Node* head;
Node* tail;
int length;

DoublyLinkList() {
head = nullptr;
tail = nullptr;
length = 0;
}

void insert(int element) {
Node* n = new Node(element);

if (head == nullptr) {
head = n;
tail = n;
} else {
tail->next = n;
n->prev = tail;
tail = n;
}
length++;
```

```
}

void splitList(DoublyLinkList& evenList, DoublyLinkList& oddList) {
Node* curr = head;
while (curr != nullptr) {
if (curr->data % 2 == 0) {
evenList.insert(curr->data);
} else {
oddList.insert(curr->data);
}
curr = curr->next;
}
}

void display() {
Node* curr = head;
while (curr != nullptr) {
cout << curr->data << "->";
curr = curr->next;
}
cout << "NULL" << endl;
}
};

int main() {
DoublyLinkList originalList;

cout << "Enter the number of elements: ";
int n;
cin >> n;

cout << "Enter " << n << " elements: ";

for (int i = 0; i < n; i++) {
int elements;
cin >> elements;
originalList.insert(elements);
}

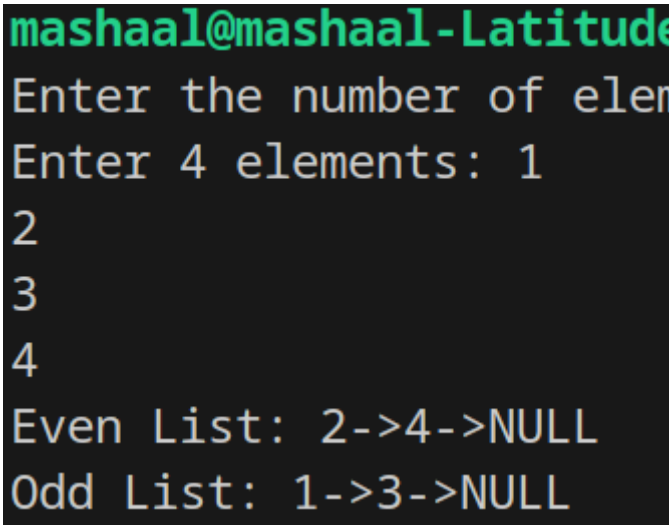
DoublyLinkList evenList;
DoublyLinkList oddList;

originalList.splitList(evenList, oddList);

cout << "Even List: ";
evenList.display();

cout << "Odd List: ";
oddList.display();

return 0;
}
```



QUESTION#3

CODE:

```
#include<iostream>
#include<string>
using namespace std;

class Node
{
public:
string data;
Node* next;
Node* prev;

Node(string song)
{
data = song;
next = nullptr;
prev = nullptr;
}
};

class Playlist
{
Node* head;
int length;
```

```
public:
Playlist()
{
head = nullptr;
length = 0;
}

~Playlist() {
if (head != nullptr) {
Node* curr = head->next;
while (curr != head) {
Node* temp = curr;
curr = curr->next;
delete temp; //we will delete the middle nodes first so that list is not dangling
}
head->next = nullptr;
delete head;
}
}

void insert(string song)
{
Node* n = new Node(song);
if (head == nullptr)
{
head = n;
head->next = head;
head->prev = head;
}
else
{
Node* tail = head->prev;

n->next = head;
n->prev = tail;
tail->next=n;
head->prev=n;
// head= n;

}
length++;
}


void remove()
{
if (head == nullptr)
{
cout << "Playlist is empty" << endl;
return;
}
```

```
if (head->next == head && head->prev == head)
{
delete head;
head=nullptr;
return;
}
else {
Node* tail = head->prev;
Node* temp = tail->prev;
temp->next = head;
head->prev = temp;
delete tail;
}
length--;
}

void display()
{
if (head == nullptr) {
cout << "Playlist is empty" << endl;
return;
}
Node* curr = head;
do {
cout << curr->data << " ";
curr = curr->next;
} while (curr != head);

cout << endl;
}

};

int main()
{
Playlist playlist;
int choice;

while (true)
{
cout <<"___SONG PLAYLIST___" "\n1. Add Song at Beginning\n"<< "2. Remove Song at End\
n"<< "3. Display Playlist\n"<< "4. Exit\n";
cout << "Enter your choice: ";
cin >> choice;

if (choice == 1)
{
string song;
cout << "Enter the name of the song: ";
cin>>song;
playlist.insert(song);
cout << "Song added to the beginning of the playlist.\n";
```

```
}
else if (choice == 2)
{
playlist.remove();
cout << "Last song removed from the playlist.\n";
}
else if (choice == 3)
{
cout << "Playlist: ";
playlist.display();
}
else if (choice == 4)
{
cout << "Exiting...\n";
return 0;
}
else
{
cout << "Invalid choice. Please try again.\n";
}
}

return 0;
}
```

```
___SONG PLAYLIST___
1. Add Song at Beginning
2. Remove Song at End
3. Display Playlist
4. Exit
Enter your choice: 3
Playlist: a b
___SONG PLAYLIST___
1. Add Song at Beginning
2. Remove Song at End
3. Display Playlist
4. Exit
Enter your choice: 2
Last song removed from the play
___SONG PLAYLIST___
1. Add Song at Beginning
2. Remove Song at End
3. Display Playlist
4. Exit
Enter your choice: 3
Playlist: a
___SONG PLAYLIST___
1. Add Song at Beginning
```